Computing Systems and Programming (5): Timeout Event

Dr. Clifford Choy

School of Design

16 Oct 2026

Timeout Event

- Allow program to execute a **handler function** AFTER a duration, or AT a specific time

- Related functions/methods
  - `enableScheduler(start_time, interval, hdlr, max_timeout_num)`
  - `setTimeoutByRelTime(evtID, hdlr, relTime, invokeIfLE0 )`
  - `setTimeoutByCount(evtID, hdlr, timeoutCount, invokeIfLE0 )`
  - `setTimeoutByAbsTime(evtID, hdlr, absTime, invokeIfLE0)`
  - `isTimeoutActive(evtID)`
  - `stopTimeout(evtID)`

iocontrol/
iocontrol_setTimeoutByRelTime

```
11  //       none
12  //
13  #include <IO_Control.h>
14
15  DeclareHandler(doSomething,_ctrl,IOControl,{
16    Serial.print("doSomething at t=");
17    Serial.println(_ctrl.getUpdateTime());
18  });
19
20  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
21    //  hdlr - name of the handler function
22    //  _ctrl - Control object
23    //  IOControl - class of object of this handler
24    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
25    //
26
27    if (!_ctrl.isTimeoutActive(0))
28    { Serial.print("start timeout for doSomething at t=");
29      Serial.println(_ctrl.getUpdateTime());
30      _ctrl.setTimeoutByRelTime(0,doSomething,2000,true);
31    }
32  });
33
34  void setup() {
35    // put your setup code here, to run once:
36    IO_Control.begin();
37
38    Serial.begin(115200);
39    Serial.println("chip started");
40
41    IO_Control.enableScheduler(100, 1, hdlr);
42      //  100 - start time of hdlr
43      //  1 - interval between consecutive invocation of hdlr
44      //  hdlr - must be declared using DeclareHandler(...)
45  }
46
47  void loop() {
48    // put your main code here, to run repeatedly:
49  }
```

Default to at most **one** timeout event if `enableScheduler(...)` has no 4-th argument

Handler function `hdlr` to be invoked by scheduler repeatedly, once every 1ms

`true` if NO currently active timeout event for `evtID = 0`

Start a timeout event, using `evtID = 0` (event ID)
Execute the handler function `doSomething` after 2000ms

Handler function `doSomething` for timeout event

Execution Logic

- `hdlr` will be executed every 1ms
- Statements in lines 28 to 30 will NOT be executed once the timeout event is started (by the `setTimeoutByRelTime(...)`)
- After `doSomething` is invoked, no more timeout event is active, and hence statements in lines 28 to 30 will be executed again, which then starts a new timeout event
- Hence, `doSomething` will be run after 2000ms again



iocontrol/
iocontrol_fsm_setTimeout_serialin

iocontrol_fsm_setTimeout_serialin

- Pre-requisite:
  - iocontrol/iocontrol_serialout_fsm1
  - Iocontrol/iocontrol_serialin

```
1   //
2   //  for demonstrating use of timeout in FSM with the following logic:
3   //    IOControl has two timers (0 and 1)
4   //    FSM has four states, and with different duration at each state (1000ms, 1500ms, 2000ms, and 2500ms)
5   //    timer 0 is for controlling timeout of each state of FSM
6   //    tiemr 1 has timeout of 500ms and for printoug out current state of FSM
7   //    when user enter "s" through Serial Monitor, the current state will be stopped immediately and move on to the next state
8   //
9   //  this example demonstrates use of the following methods:
10  //      enableScheduler(start_time,interval,hdlr,max_timer_num)
11  //      setTimeoutByRelTime(..)
12  //      isTimeoutActive()
13  //      stopTimeout()
14  //
```

```
109  void setup() {
110    // put your setup code here, to run once:
111    IO_Control.begin();
112
113    Serial.begin(115200);
114    Serial.println("chip started");
115
116    IO_Control.enableScheduler(100, 1, hdlr, 2);
117      //  100 - start time of hdlr
118      //  1 - interval between consecutive invocation of hdlr
119      //  hdlr - must be declared using DeclareHandler(...)
120      //  2 - maximum no of timeout event = 2
121  }
```

4-th argument for `enableScheduler(...)` to specify how many timeout event it supports simultaneously

Default value is 1

Timeout event ID `0` for setting next state of FSM
Handler function `setNextState` is invoked immediately
Line 89 will be executed for one time ONLY

Timeout event ID `1` for setting next state of FSM
Handler function `printCurrentState` is invoked immediately
Line 92 will be executed for one time ONLY

When user entered “s”, timeout event 0 is stopped, and then restarted, which invoke `setNextState` immediately

```
76  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
77    //  hdlr - name of the handler function
78    //  _ctrl - Control object
79    //  IOControl - class of object of this handler
80    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
81    //
82
83    // if (_ctrl.isSchedulerTimeEvent())
84    //     Serial.println("IOControl - SchedulerTimerEvent");
85    // Serial.print("current time=");
86    // Serial.println(_ctrl.getUpdateTime());
87
88    if (!_ctrl.isTimeoutActive(0))
89    { _ctrl.setTimeoutByRelTime(0,setNextState,0,true);
90    }
91    if (!_ctrl.isTimeoutActive(1))
92    { _ctrl.setTimeoutByRelTime(1,printCurrentState,0,true);
93    }
94
95    if (Serial.available() > 0) {
96      String input = Serial.readStringUntil('\n');
97      if (input.equals("s"))
98      { //  stop current state and move to next
99        Serial.print("stop current state=");
100       Serial.print(state);
101       Serial.print(" at t=");
102       Serial.println(_ctrl.getUpdateTime());
103       _ctrl.stopTimeout(0);
104       _ctrl.setTimeoutByRelTime(0,setNextState,0,true);
105     }
106   }
107 });
```

setNextState

- Whenever this handler function is invoked, the state variable for the FSM will be advanced to the next state AND timeout event `0` will be restarted to wait for a duration corresponding to that of the new state

- For example, state `0` has a duration of 1000ms

```
34  DeclareHandler(setNextState,_ctrl,IOControl,{
35    switch (state)
36    { case -1:
37      case 3:
38        state=0;
39        _ctrl.setTimeoutByRelTime(0,setNextState,1000,true);
40        Serial.print("entering state=");
41        Serial.print(state);
42        Serial.print(" at t=");
43        Serial.print(_ctrl.getUpdateTime());
44        Serial.println(", duration = 1000ms");
45        break;
46      case 0:
47        state=1;
48        _ctrl.setTimeoutByRelTime(0,setNextState,1500,true);
49        Serial.print("entering state=");
50        Serial.print(state);
51        Serial.print(" at t=");
52        Serial.print(_ctrl.getUpdateTime());
53        Serial.println(", duration = 1500ms");
54        break;
55      case 1:
56        state=2;
57        _ctrl.setTimeoutByRelTime(0,setNextState,2000,true);
58        Serial.print("entering state=");
59        Serial.print(state);
60        Serial.print(" at t=");
61        Serial.print(_ctrl.getUpdateTime());
62        Serial.println(", duration = 2000ms");
63        break;
64      case 2:
65        state=3;
66        _ctrl.setTimeoutByRelTime(0,setNextState,2500,true);
67        Serial.print("entering state=");
68        Serial.print(state);
69        Serial.print(" at t=");
70        Serial.print(_ctrl.getUpdateTime());
71        Serial.println(", duration = 2500ms");
72        break;
73    }
74  });
```

printCurrentState

- Run repeatedly every 500ms
- For printing current state of the FSM

```
26  DeclareHandler(printCurrentState,_ctrl,IOControl,{
27    Serial.print("current state=");
28    Serial.print(state);
29    Serial.print(" at time t=");
30    Serial.println(_ctrl.getUpdateTime());
31    _ctrl.setTimeoutByRelTime(1,printCurrentState,500,true);
32  });
```

Immediately start another timeout event (evtID=1) when `printCurrentState` is invoked

Output Serial Monitor ×

Message (Enter to send message to 'AirM2M_CORE_ESP32C3' on 'COM

```
chip started
current state=-1 at time t=101
entering state=0 at t=101, duration = 1000ms
current state=0 at time t=601
current state=0 at time t=1101
entering state=1 at t=1101, duration = 1500ms
current state=1 at time t=1601
current state=1 at time t=2101
current state=1 at time t=2601
entering state=2 at t=2601, duration = 2000ms
current state=2 at time t=3101
current state=2 at time t=3601
current state=2 at time t=4101
current state=2 at time t=4601
entering state=3 at t=4601, duration = 2500ms
current state=3 at time t=5101
current state=3 at time t=5601
current state=3 at time t=6101
current state=3 at time t=6601
current state=3 at time t=7101
entering state=0 at t=7101, duration = 1000ms
current state=0 at time t=7601
current state=0 at time t=8101
entering state=1 at t=8101, duration = 1500ms
current state=1 at time t=8601
current state=1 at time t=9101
current state=1 at time t=9601
entering state=2 at t=9601, duration = 2000ms
current state=2 at time t=10101
current state=2 at time t=10601
current state=2 at time t=11101
current state=2 at time t=11601
entering state=3 at t=11601, duration = 2500ms
current state=3 at time t=12101
```

FSM went from state 0 to 3 and then went back to `state == 0`, without interruption, when user did not enter anything

User entered “s”, which stopped the current state (at `state == 2`)

`setNextState` was invoked, which changed from `state == 2` to `state == 3` at the time when the state was stopped (with 1ms delay)

Output Serial Monitor ×

Message (Enter to send message to 'AirM2M_CORE_ESP32C3' on 'COM12')

```
current state=3 at time t=69601
current state=3 at time t=70101
entering state=0 at t=70101, duration = 1000ms
current state=0 at time t=70601
current state=0 at time t=71101
entering state=1 at t=71101, duration = 1500ms
current state=1 at time t=71601
current state=1 at time t=72101
current state=1 at time t=72601
entering state=2 at t=72601, duration = 2000ms
stop current state=2 at t=72615
current state=2 at time t=73616
entering state=3 at t=73616, duration = 2500ms
current state=3 at time t=74116
current state=3 at time t=74616
current state=3 at time t=75116
current state=3 at time t=75616
current state=3 at time t=76116
entering state=0 at t=76116, duration = 1000ms
current state=0 at time t=76616
current state=0 at time t=77116
entering state=1 at t=77116, duration = 1500ms
current state=1 at time t=77616
current state=1 at time t=78116
current state=1 at time t=78616
entering state=2 at t=78616, duration = 2000ms
current state=2 at time t=79116
current state=2 at time t=79616
current state=2 at time t=80116
current state=2 at time t=80616
entering state=3 at t=80616, duration = 2500ms
current state=3 at time t=81116
current state=3 at time t=81616
```

Task 5

- Modify `iocontrol/iocontrol_fsm_setTimeout_serialin` so that user can change the duration of each state in the FSM by entering command through Serial Monitor. Specifically, user can enter command in the form:

  <state>,*<new duration>*

- where *<state>* is the state number (from 0 to 3), and *<new duration>* is the new duration of the state
- For example, if user entered

  0,1200

- then duration of `state == 0` will become 1200ms

- Under the heading “Task 5” in your submission page, upload the following:
  - Video recording of the output from Serial Monitor demonstrating the above logic
  - Compressed file of your sketch (in zip format)

iocontrol/iocontrol_setTimeout_wc

- An example for demonstrating other `setTimeout` functions
- Activated through commands sent from Serial Monitor

```
//
//  For demonstrating how to use methods relating to timeout, including:
//    enableScheduler(start_time,interval,hdlr,max_timer_num)
//    setTimeoutByCount(..)
//    setTimeoutByRelTime(..)
//    setTimeoutByAbsTime(..)
//    isTimeoutActive()
//    stopTimeout()
//
//  board:
//    Luatos ESP32C3 board
//    ESP32S3 Dev Module
//
//  commands:
//  c             - check whether any active timneout event
//  c,<count>     - start timeout by count
//  r,<rel_time>  - start timeout by relative time (in ms)
//  a,<abs_time>  - start timeout by absolute time (in ms)
//  ar,<rel_time> - start timeout by absolute time (in ms), at current time + <rel_time> (<rel_time> can be -ve)
//  t             - get current time (in ms)
//  s             - stop an active timeout event
//
//  examples:
//    c,1000 - timeout by count, with total timeout duration = 1000(count)*1(interval)
//    r,2000 - timeout by rel time, with total timeout duration = 2000ms
//    a,3000 - timeout by abs time, with absolute time when timeout event triggered = 3000ms
//    ar,-1000 - timeout by abs time = current time - 1000 ms
//
```

Summary

- `enableScheduler(start_time, interval, hdlr, max_timeout_num)`
- `setTimeoutByRelTime(evtID, hdlr, relTime, invokeIfLE0 )`
- `setTimeoutByCount(evtID, hdlr, timeoutCount, invokeIfLE0 )`
- `setTimeoutByAbsTime(evtID, hdlr, absTime, invokeIfLE0)`
- `isTimeoutActive(evtID)`
- `stopTimeout(evtID)`